GIT ASSIGNMENT- BASIC GIT COMMANDS

1) **Initialise Git repository :**

```
PS C:\Users\user\Desktop\GitLab> git --version
git version 2.55.0.windows.2
PS C:\Users\user\Desktop\GitLab>
```

```
user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master)
$ git init
Initialized empty Git repository in C:/Users/user/Desktop/GitLab/.git/
```

i)“ git init” -> Initializes a new Git repository in the project directory

2) **Configure Git username and email :**

```
user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master)
$ git config user.name "Aayush Pupalwad"

user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master)
$ git config user.email "2025ucp1019@mnit.ac.in"

user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master)
$ git config user.name
Aayush Pupalwad

user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master)
$ git config user.email
2025ucp1019@mnit.ac.in
```

i)Configures the username and email used for commits.

3) Create and stage `readme.txt` :

i) `git add ->` Stages files or changes for the next commit.

ii) `git status ->` Displays the current state of the working directory and staging area.

```
user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master)
$ git add readme.txt

user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   readme.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        vecadd.py
```

.

4) Commit the first file

```
user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        vecadd.py

nothing added to commit but untracked files present (use "git add" to track)
```

`i)git commit ->` Saves staged changes as a new commit in the repository history.

5) Modify the README and use `git diff` :

```
user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master)
$ git diff
diff --git a/readme.txt b/readme.txt
index e75535e..43ed133 100644
--- a/readme.txt
+++ b/readme.txt
@@ -1,2 +1,3 @@
 GitLab is a sample project for learning Git commands.
-It contains a Python program that calculates the dot product of two vectors.
\ No newline at end of file
+It contains a Python program that calculates the dot product of two vectors.
+This project demonstrates basic Git version control operations.
\ No newline at end of file
```

`i)git diff ->` Displays unstaged differences between the working directory and the index.

6) Explore `git restore` :

```
user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master)
$ git restore readme.txt

user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        vecadd.py

nothing added to commit but untracked files present (use "git add" to track)

user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master)
$ git diff
```

```
user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master)
$ git restore --staged readme.txt

user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   readme.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        vecadd.py

no changes added to commit (use "git add" and/or "git commit -a")
```

`i)git restore ->` Discards unstaged changes to a file.

`ii)git restore --staged ->` Removes staged changes from the index while preserving working-directory modifications.

7)View commit history using `git log` :

`i)git log ->Displays the repository's commit history.`

```
user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master)
$ git log
commit 8f8f9aed4f177311b4f453ce52ed26c28b9d670d (HEAD -> master)
Author: Aayush Pupalwad <2025ucp1019@mnit.ac.in>
Date:   Thu Oct 29 21:58:11 2026 +0530

    Update README description

commit e2cfd7714e86a174619bdac28133526a5992bda2
Author: Aayush Pupalwad <2025ucp1019@mnit.ac.in>
Date:   Thu Oct 29 21:43:08 2026 +0530

    Adding first file.
```

8) Revert a previous commit :

i)git revert -> Creates a new commit that reverses the changes introduced by an earlier commit.

```
user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master)
$ git revert e2cfd77
CONFLICT (modify/delete): readme.txt deleted in (empty tree) and modified in HEAD.  Version HEAD of
readme.txt left in tree.
error: could not revert e2cfd77... Adding first file.
hint: After resolving the conflicts, mark them with
hint: "git add/rm <pathspec>", then run
hint: "git revert --continue".
hint: You can instead skip this commit with "git revert --skip".
hint: To abort and get back to the state before "git revert",
hint: run "git revert --abort".
hint: Disable this message with "git config set advice.mergeConflict false"
```

9) Commit vecadd.py :

Commit the vecadd.py using first git commit.
Then you can see the commit history by git log.

```
user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master|REVERTING)
$ git commit -m "Add vector dot product program"
U       readme.txt
error: Committing is not possible because you have unmerged files.
hint: Fix them up in the work tree, and then use 'git add/rm <file>'
hint: as appropriate to mark resolution and make a commit.
fatal: Exiting because of an unresolved conflict.

user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master|REVERTING)
$ git log
commit 8f8f9aed4f177311b4f453ce52ed26c28b9d670d (HEAD -> master)
Author: Aayush Pupalwad <2025ucp1019@mnit.ac.in>
Date:   Thu Oct 29 21:58:11 2026 +0530

    Update README description

commit e2cfd7714e86a174619bdac28133526a5992bda2
Author: Aayush Pupalwad <2025ucp1019@mnit.ac.in>
Date:   Thu Oct 29 21:43:08 2026 +0530

    Adding first file.
```

10)Remove and commit the deletion of readme.txt :

```
user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master|REVERTING)
$ git commit -m "Remove README file"
[master d2d98f3] Remove README file
 2 files changed, 15 insertions(+), 3 deletions(-)
 delete mode 100644 readme.txt
 create mode 100644 vecadd.py

user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master)
$ git status
On branch master
nothing to commit, working tree clean

user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master)
$ git log --oneline
d2d98f3 (HEAD -> master) Remove README file
8f8f9ae Update README description
e2cfd77 Adding first file.
```

i) git rm -> Removes a tracked file and stages its deletion.

```
user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master|REVERTING)
$ git rm readme.txt
rm 'readme.txt'

user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master|REVERTING)
$ git staus
git: 'staus' is not a git command. See 'git --help'.

The most similar command is
        status

user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master|REVERTING)
$ git status
On branch master
You are currently reverting commit e2cfd77.
  (all conflicts fixed: run "git revert --continue")
  (use "git revert --skip" to skip this patch)
  (use "git revert --abort" to cancel the revert operation)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    readme.txt
        new file:   vecadd.py
```

11)Check out an earlier commit :

i) git checkout <commit-hash> ->Checks out a particular commit, allowing you to inspect an earlier version.

```
user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab ((d2d98f3...))
$ git log --oneline
d2d98f3 (HEAD, master) Remove README file
8f8f9ae Update README description
e2cfd77 Adding first file.

user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab ((d2d98f3...))
$ git status
HEAD detached at d2d98f3
nothing to commit, working tree clean
```

```
$ git log --oneline
d2d98f3 (HEAD -> master) Remove README file
8f8f9ae Update README description
e2cfd77 Adding first file.

user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master)
$ git log --oneline
d2d98f3 (HEAD -> master) Remove README file
8f8f9ae Update README description
e2cfd77 Adding first file.

user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master)
$ git checkout d2d98f3
Note: switching to 'd2d98f3'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:
```

12) Return to the latest commit :

```
user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab ((d2d98f3...))
$ git branch
* (HEAD detached at d2d98f3)
  master

user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab ((d2d98f3...))
$ git checkout master
Switched to branch 'master'

user@LAPTOP-SJDDE444 MINGW64 ~/Desktop/GitLab (master)
$ git status
On branch master
nothing to commit, working tree clean
```

i)git checkout <branch-name> ->Switches back to the specified branch.

Like in this example we have switched back to the master branch.